



Training Python-Django - 1

Librairies

*Summary: Today, we're gonna learn how to handle some librairies that might come handy in **Python**.*

Version: 1.3

Contents

I	Preamble	2
II	General rules	3
III	Today's Specific Rules	4
IV	Exercise 00	5
V	Exercise 01	6
VI	Exercise 02	8
VII	Exercise 03	10
VIII	Exercise 04	13
IX	Exercise 05	14
X	Submission and peer-evaluation	15

Chapter I

Preamble

Geohashing From Wikipedia, the free encyclopedia

Geohashing is an outdoor recreational activity inspired by the xkcd webcomic, in which participants have to reach a random location (chosen by a computer algorithm), prove their achievement by taking a picture of a Global Positioning System (GPS) receiver or another mobile device and then tell the story of their trip online. Proof based on non-electronic navigation is also acceptable.

Whereas other outdoor recreational activities like geocaching have a precise goal, geohashing is mainly fueled by its pointlessness, which is deemed funny by its players. The resulting geohashing community and culture is thus extremely tongue-in-cheek, supporting any kind of humorous behavior during the practice of geohashing and resulting in a parody of traditional outdoor activities. Navigating to a random point need not be pointless. Some geohashers document new mapping features they find on the OpenStreetMap project.

Source [Wikipedia](#)

Chapter II

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a 0 during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.



For obvious security reasons, any credentials, API keys, env variables etc... must be saved locally in a `.env` file and ignored by git. Publicly stored credentials will lead you directly to a failure of the project.

Chapter III

Today's Specific Rules

- No code in the global scope. Make functions!
- Each turned-in file must end with a function call in a condition identical to:

```
if __name__ == '__main__':  
    your_function( whatever, parameter, is, required )
```

- You can set up error management in this condition.
- Imports are prohibited, except for those specified in the "Authorized Functions" section in each exercise's header.
- You will use the `python3` interpreter.

Chapter IV

Exercise 00

	Exercise00
Exercise 00: Antigravity	
Directory: <i>ex00/</i>	
Files to Submit: <code>geohashing.py</code>	
Authorized: modules <code>sys</code> and <code>antigravity</code>	

This is a little warm-up exercise echoing today's preamble. Nothing too complicated.

Create a little program named `geohashing.py` that will take as many parameters as necessary to calculate a typical geohash and should obviously calculate this geohash before displaying it on the standard output.

In case of an error, the program must display a relevant message that you have chosen before exiting properly.

This schematic might help: [Geohashing algorithm](#).

Chapter V

Exercise 01

	Exercise01
Exercise 01: Pip	
Directory: <i>ex01/</i>	
Files to Submit: <code>my_script.sh</code> <code>my_program.py</code>	
Authorized: module <code>path</code> (previously called <code>path.py</code>)	

`path` is a library that implements a `Path` object around Python's `os.path` module, making its use very intuitive.

In this exercise, you will create a **bash** script that installs this library, as well as a **Python** program that uses it.

The Shell script must fit this description:

- Its name must have a `.sh` extension because it's a **Shell** script.
- It must display which pip version it uses.
- It must install the `path.py` development version from its **GitHub** repository into a folder named `local_lib`, located in the repository folder. If the library has already been installed in the folder, the installation must overwrite it.
- It must write the `path` installation logs to a file with a `.log` extension.
- If the library has been properly installed, it must execute the small program you have created.

The `Python` program you must create is a composition of your choice that must, however, observe these constraints:

- Its extension must be `.py` because it's a `Python` program.
- It must import the `path` module from the location where this library has been installed by the previous script.
- It must create a folder and a file inside this folder, write something to this file, and then read and display its content.
- It must respect the specific rules of the day.

Chapter VI

Exercise 02

	Exercise02
Exercise 02: Request an API	
Directory: <i>ex02/</i>	
Files to Submit: <code>request_wikipedia.py</code> <code>requirements.txt</code>	
Authorized: modules <code>requests</code> , <code>json</code> , <code>dewiki</code> and <code>sys</code>	

Wikipedia is an amazing collaborative tool that you certainly know. It's available in your favorite browser and as a mobile application. You're invited to create a tool that will allow you to query this now-essential website directly from your terminal.

To do so, you must design a program named `request_wikipedia.py` which takes a `string` as a parameter and performs a search via the [Wikipedia API](#) before writing the result to a file. You can query either the French or English API.

- The program must write a result, even if the request is misspelled. Take the original website as an example: if it finds a result for a given request, your program should too.
- The result must not be formatted in JSON or Wiki Markup before being written to the file.
- The name of the file must be formatted like this: `name_of_the_search.wiki` and must not contain any spaces.
- In case of a missing parameter, an invalid parameter, an invalid request, information not found, a server problem, or any other problem: no file should be created and a relevant error message must be displayed on the console.
- Include a `requirements.txt` file in your repository. It will be used during your evaluation to install the libraries necessary for your `VirtualEnv` program or on the system.



The dewiki library is not perfect. We're not looking for the best result; this is not the goal of this exercise.



Carefully read the API documentation. Pay attention to the structure returned by the API.

Here is an example of what's expected:

```
$>python3 request_wikipedia.py "chocolatine"
$>cat chocolatine.wiki

Une chocolatine designe :
* une viennoiserie au chocolat, aussi appelee pain au chocolat ou couque au chocolat ;
* une viennoiserie a la creme patissiere et au chocolat, aussi appelee suisse ;
* une sorte de bonbon au chocolat ;
* un ouvrage d'Anna Rozen

Malgre son usage ancien, le mot n'est entre dans le dictionnaire Petit Robert qu'en 2007 et dans le
Petit Larousse qu'en 2011.

L'utilisation du terme "Chocolatine" se retrouve egalement au Quebec, dont la langue a evolue a partir
du vieux francais differemment du francais employe en Europe, mais cet usage ne prouve ni n'
infirme aucune anteriorite, dependant du hasard de l'usage du premier commercant l'ayant introduit
au Quebec.

References

Categorie:Patisserie
Categorie:Chocolat
```

Chapter VII

Exercise 03

	Exercise03
Exercise 03: HTML Parser	
Directory: <i>ex03/</i>	
Files to Submit: <code>roads_to_philosophy.py</code> , <code>requirements.txt</code>	
Authorized: modules <code>sys</code> , <code>requests</code> and <code>BeautifulSoup</code>	

Legend has it that if you start from any Wikipedia article, click on the first link in its introduction that is neither in italics nor between brackets, and repeat the process ad libitum, you will always end up on the article about Philosophy.

Well, this is not a legend! (Please, look astounded.) As this [Wikipedia article](#) can testify.

But since you only believe what you see with your own eyes, you **must** create a program that tests this phenomenon by listing and counting all the articles between your request and the Wikipedia article: the **roads to philosophy**.

This program must be named `roads_to_philosophy.py` and behave as follows:

- The program must take a string as a parameter that is a word or a group of words matching only one Wikipedia search result.
- The program must request an **English** Wikipedia URL identical to a standard search in a browser. In other words: you **cannot** use the site's API.

- It must parse the HTML page using the BeautifulSoup library to:
 - Find any redirection and take it into account in the **roads to philosophy**. Beware, it is not a URL redirection.
 - Find the main title of the page and add it to the **roads to philosophy**.
 - Find (if it exists) **the first link in the introductory paragraph** leading to another Wikipedia article. Rather than ignoring what is in italics or between brackets, the program must carefully **ignore** links that do not lead to a new article, such as those leading to the help section of Wikipedia.
- The program must start again from **step 2** using the link obtained during the previous step until reaching one of the following occurrences:
 - The link leads to the Philosophy page. If so, it must print the visited articles as well as the total count of these articles in the following format: `<number> roads from <request> to philosophy` on the standard output.
 - The page does not include **any valid link**. The program must display: `It leads to a dead end !`.
 - The link leads to a page that has already been visited, which means the program is about to loop indefinitely. If so, the displayed message must be: `It leads to an infinite loop !`
- At this stage, after displaying the necessary messages on the standard output, the program must exit properly.



If, at any time during the execution of the program, a connection, server, parameter, request, or any other kind of error occurs, the program must exit properly with a relevant error message.

Like the previous exercise, you must provide a `requirements.txt` file with your program to facilitate library installation.

Your program's output must look like this:

```
$> python3 roads_to_philosophy.py "42 (number)"
42 (number)
Natural number
Mathematics
Ancient Greek
Greek language
Modern Greek
Colloquialism
Word
Linguistics
Science
Knowledge
Awareness
Conscious
Consciousness
Quality (philosophy)
Philosophy
17 roads from 42 (number) to philosophy !
$> python3 roads_to_philosophy.py Accuvio
It leads to a dead end !
$>
```



The Wikipedia community often updates articles. It is highly probable that between the creation of this subject and the day you take it, the roads to philosophy have changed and the Accuvio example is no longer a dead end.

Chapter VIII

Exercise 04

	Exercise04
Exercise 04: Virtualenv	
Directory: <i>ex04/</i>	
Files to Submit: <code>requirement.txt</code> <code>my_script.sh</code>	
Authorized: <code>everything</code>	

Tomorrow, you'll start your Django framework training. You must lay the foundation configuring an easy little installation.

TO do so, you will create two elements:

- A `requirement.txt` file that must include the latest stable versions of `django` and `psycpg2`.
- A script with the following behavior:
 - Have the `.sh` extension.
 - Create a `virtualenv` on `python3` named `django_venv`.
 - Install the `requirement.txt` file that you've created in the `VirtualEnv`.
 - The `virtualenv` must be **activated** when quitting.

Chapter IX

Exercise 05

	Exercise05
Exercise 05: Hello World	
Directory: <i>ex05/</i>	
Files to Submit: any necessary files	
Authorized: everything	

It must be frustrating to simply install Django. We feel for you.

This is why you will end this day about libraries with a bang, by designing your first Hello World application with Django.

In this final exercise, you must follow and adapt the official tutorial to create a web page that simply displays the text **Hello World!** in the browser at the following address: <http://localhost:8000/helloworld>.

Your repository must contain the Django project.

Chapter X

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.



The evaluation process will happen on the computer of the evaluated group.